

EE6406

Analytic and Ensemble Machine Learning

(Lecture 4: Learning Score Functions)

Kar-Ann Toh

School of Electrical and Electronic Engineering
Nanyang Technological University

karanntoh@ntu.edu.sg

Semester 1, AY2026-27

EE6406: Analytic and Ensemble Machine Learning

Lecture 4

Learning Score Functions

Content

- 1 Introduction
 - Motivation for Having Different Loss Functions
- 2 Regression
 - Regression Accuracy
- 3 Classification
 - Classification Accuracy
- 4 Error Metrics
 - Error Metrics and Loss Functions
- 5 Operating Characteristics
 - Ranking and Operating Characteristics
- 6 Relationships
 - Relationships Among LSE, TER and AUC

Introduction

- The previous lecture has established several learning models with particular focus on linear parametric form.
- In this lecture, such learning models will be utilized in a *score metric* to form the *learning objective function*.
- The score metric is also known as the *loss function* which measures the error of each learning instant. The *sum* of the learning scores or losses over a set of training samples forms the *learning objective function*, which is also known as the *learning criterion function* or *learning cost function*.
- Three basic types of learning objective or criterion function namely,
 - the *regression accuracy*,
 - the *classification accuracy*, and
 - the *ranking and operating characteristics*will be presented in this lecture.

Introduction

- The terms **loss function** and **error function** are sometimes used interchangeably, but there is a subtle difference between them.
 - An **error function** quantifies the deviation of a predicted value from the actual value.
 - In contrast, a **loss function** evaluates the unwanted effects of this error, assigning a penalty based on its magnitude.
- For example, in **regression**, the loss function often measures the squared error between the predicted and actual values. In **classification**, the loss function assesses how many samples are misclassified by considering the deviation between predicted and true labels.

Introduction

Learning Cost or Objective Function

$$\arg \min_{\mathbf{w}} J(\mathbf{w}) = \arg \min_{\mathbf{w}} \sum_{i=1}^m L(g(\mathbf{x}_i, \mathbf{w}), y_i) + \lambda R(\mathbf{w})$$

where

$J(\mathbf{w})$: Learning cost function (optimization criterion)

$L(\cdot)$: Loss function (also known as Score function)

$g(\mathbf{x}_i, \mathbf{w})$: Learning model

$\mathbf{x}_i$: Learning model input vector

$\mathbf{w}$: Learning parameter vector

y_i : Learning target

m : Sample size

$R(\mathbf{w})$: Regularization function

λ : Regularization factor

Introduction

Building Blocks of Learning Algorithms

- **Learning model g** reflects our belief about the relationship between the features $\mathbf{x}_i$ and target y_i
- **Loss function L** is the penalty for predicting $g_{\mathbf{w}}(\mathbf{x}_i)$ (or $g(\mathbf{x}_i, \mathbf{w})$) when the true value is y_i
- **Regularization R** encourages less complex models
- **Cost function J** is the final optimization criterion we want to minimize
- **Optimization routine** to find solution to cost function

Motivation for having different loss functions

Two common loss functions:

- Regression: **continuous** target value
- Classification: **discrete** target label

We have seen the following cost/criterion function for optimization thus far:

$$\min_{\mathbf{w}} \sum_{i=1}^m (g(\mathbf{x}_i, \mathbf{w}) - y_i)^2,$$

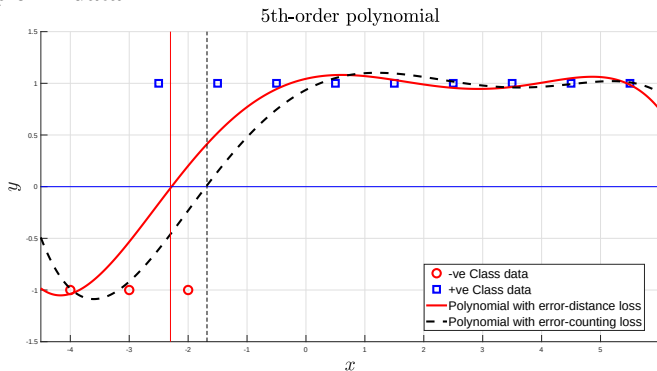
which measures the sum of **error distances**.

However, for classification, we might need a **discrete counting** loss such as

$$\min_{\mathbf{w}} \frac{1}{2} \sum_{i=1}^m [1 - \text{sgn}(y_i g(\mathbf{x}_i, \mathbf{w}))].$$

Motivation for having different loss functions

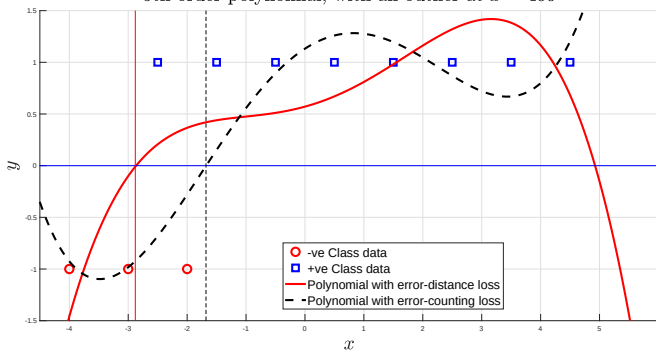
Example 1D data



Motivation for having different loss functions

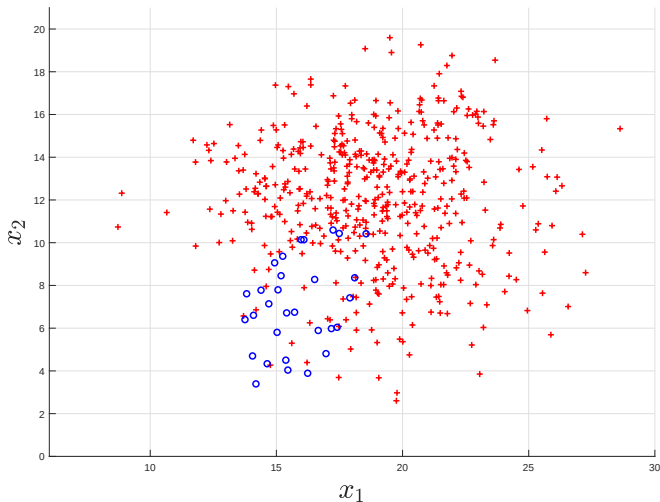
Example 1D data

5th-order polynomial, with an outlier at $x = 400$



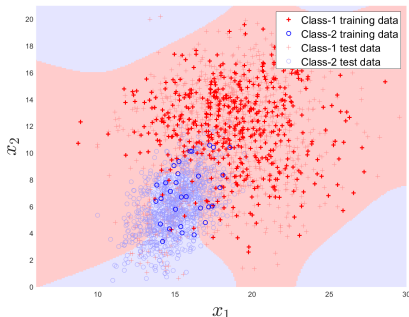
Motivation for having different loss functions

Example 2D data

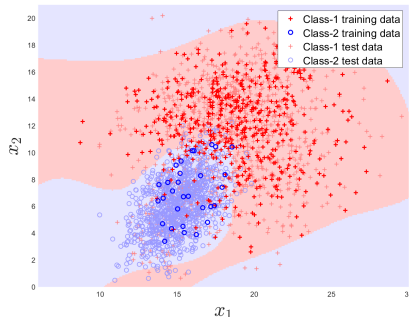


Motivation for having different loss functions

Example 2D data (cont'd)



An 8th-order polynomial trained using $\min_{\mathbf{w}} \sum_{i=1}^m (g(\mathbf{x}_i, \mathbf{w}) - y_i)^2$

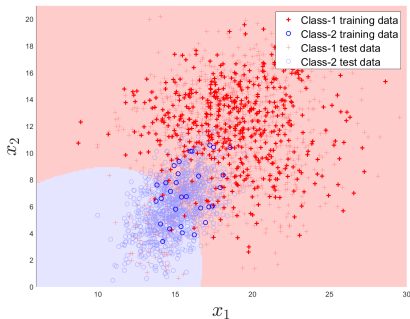


An 8th-order polynomial trained using a 'discrete counting based' cost function

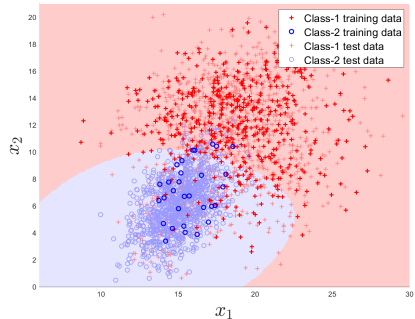
Motivation for having different loss functions

Example 2D data (cont'd):

Inclusion of an outlier point for class-1 at (1000,1000) for training



An 8th-order polynomial trained using $\min_{\mathbf{w}} \sum_{i=1}^m (g(\mathbf{x}_i, \mathbf{w}) - y_i)^2$



An 8th-order polynomial trained using a 'discrete counting based' cost function

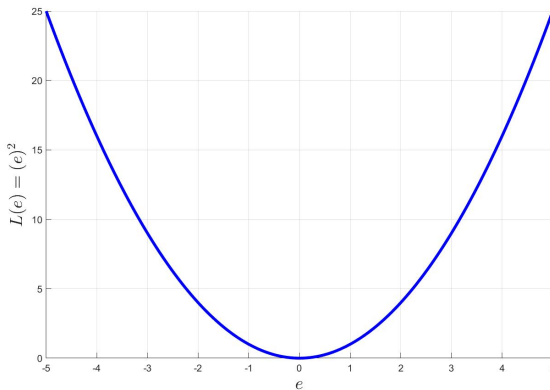
Common Loss Functions

- Squared error loss (we have seen this several times)
- Binary loss
- Logistic loss
- Hinge loss
- Exponential loss
- Cross-entropy loss

Squared error loss

- Also known as **quadratic loss function**

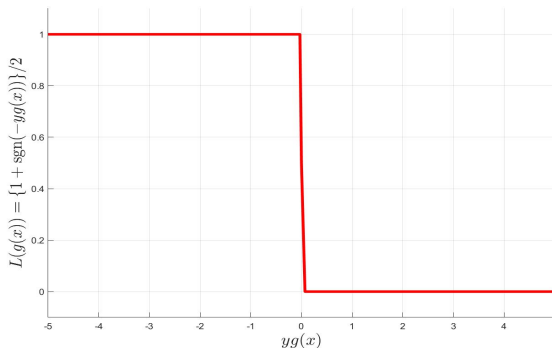
$$L(e) = (e)^2 = (g(\mathbf{x}, \mathbf{w}) - y)^2$$



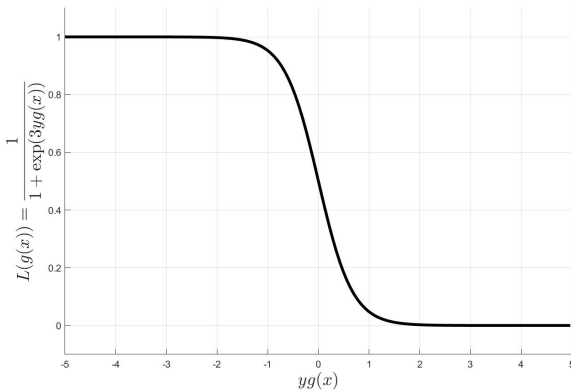
Binary loss

- 1 when $g(\mathbf{x}, \mathbf{w})$ and y are different
- 0 when they are the same
- Also known as **0-1 loss**

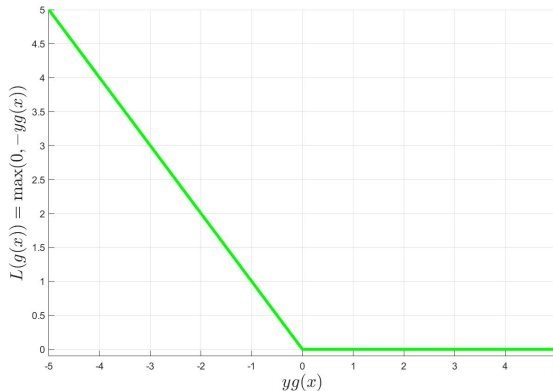
$$L(g(\mathbf{x}, \mathbf{w})) = [1 + \text{sgn}(-yg(\mathbf{x}, \mathbf{w}))] / 2, y \in \{-1, +1\}$$



$$\text{Logistic loss: } L(g(x, w)) = \frac{1}{1 + e^{yg(x, w)}}$$



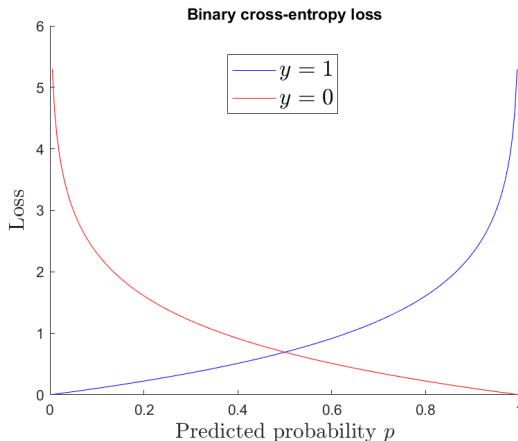
Hinge loss: $L(g(x, w)) = \max(0, -yg(x, w))$



Loss Functions

Binary cross-entropy loss function

$$L(p) = -y \log p - (1 - y) \log(1 - p)$$



Regression Accuracy

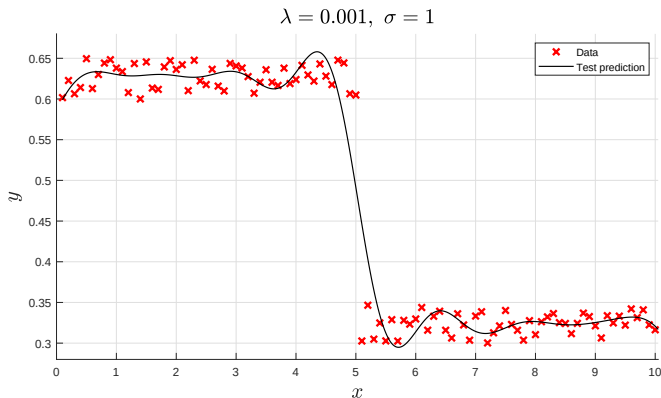
What is Regression?

In the context of machine learning, **regression** is a type of supervised learning task that involves predicting a continuous numerical value based on input features.

The goal of a regression model is to find the relationship between the **input variables** (independent variables) and the **output** (dependent variable), minimizing the difference between the predicted and actual values.

Regression Accuracy

Illustration



Regression Accuracy

The **regression accuracy** is often expressed in terms of the *mean squared error (MSE)* defined as

$$\text{MSE} : J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m [g(\mathbf{x}_i, \mathbf{w}) - y_i]^2. \quad (4.1)$$

This criterion measures the mean of the squared errors over the set of samples utilized. The learning score metric or **loss function** adopted here is the **squared error** distance given by $[g(\mathbf{x}_i, \mathbf{w}) - y_i]^2$. In terms of the minimizer solution, the scaling factor $1/m$ is immaterial where the *sum of squared errors (SSE)* (4.2) can also be used for optimization.

$$\text{SSE} : J(\mathbf{w}) = \sum_{i=1}^m [g(\mathbf{x}_i, \mathbf{w}) - y_i]^2. \quad (4.2)$$

Regression Accuracy

Example 4.1

Suppose we have m samples of $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$ with corresponding target $y \in \mathbb{R}$, then these samples can be indexed as $\mathbf{x}_i$ and y_i with $i = 1, \dots, m$. With inclusion of intercept or bias, the input samples $\mathbf{x}_i, i = 1, \dots, m$ can be packed in matrix form as

$$\mathbf{x} = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & \cdots & x_{1,d} \\ 1 & x_{2,1} & x_{2,2} & \cdots & x_{2,d} \\ \vdots & & & & \\ 1 & x_{m,1} & x_{m,2} & \cdots & x_{m,d} \end{bmatrix}. \quad (4.3)$$

Correspondingly, the target samples $y_i, i = 1, \dots, m$ can be packed as a column vector $\mathbf{y} = [y_1, y_2, \dots, y_m]^T$. Moreover, the outputs given by the learning model $g(\mathbf{x}_i, \mathbf{w}), i = 1, \dots, m$ can also be packed as $\mathbf{g} = [g(\mathbf{x}_1, \hat{\mathbf{w}}), g(\mathbf{x}_2, \hat{\mathbf{w}}), \dots, g(\mathbf{x}_m, \hat{\mathbf{w}})]^T$ for a certain parameter value given by $\hat{\mathbf{w}}$.

Regression Accuracy

Example 4.1 cont'd

Then, the SSE can be written in vector notation as

$$\text{SSE} : J(\mathbf{w}) = (\mathbf{g} - \mathbf{y})^T (\mathbf{g} - \mathbf{y}). \quad (4.4)$$

If $g(\mathbf{x}, \mathbf{w})$ is a linear model given by $\mathbf{w}^T \mathbf{x}$, then the SSE can be written as

$$\text{SSE} : J(\mathbf{w}) = (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}). \quad (4.5)$$

More generally, if $g(\mathbf{x}, \mathbf{w})$ has a linear parametric form, then the SSE can be written as

$$\text{SSE} : J(\mathbf{w}) = (\mathbf{P}\mathbf{w} - \mathbf{y})^T (\mathbf{P}\mathbf{w} - \mathbf{y}), \quad (4.6)$$

where $\mathbf{P}$ is a stacking of transformed sample row vectors $\mathbf{p}(\mathbf{x}_i) = [1, p(\mathbf{x}_1), p(\mathbf{x}_2), \dots, p(\mathbf{x}_m)]$ (see e.g., Example 3.7). These linear formulations in matrix-vector form allows utilization of matrix algebra for solving the minimization problem.

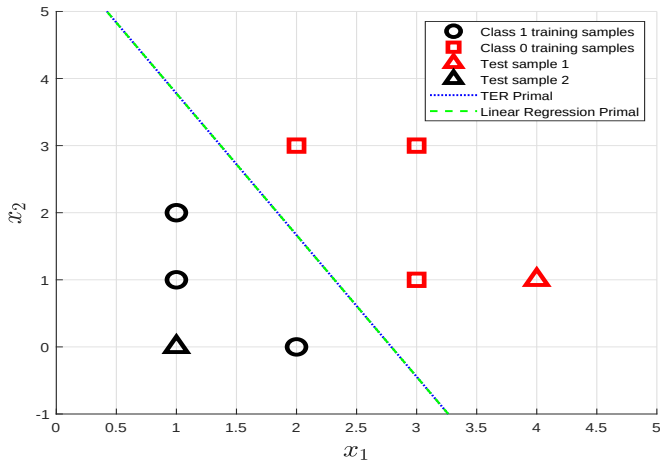
Classification Accuracy

What is Classification?

- The **classification accuracy** is different from the **regression accuracy** in the sense that the former is a discrete **counting measure** whereas the later is a **distance measure** between two scalar values.
- Before going into the several variants of the classification accuracy, we shall begin with the types of classification outcomes.
- For **binary classification** of labels '0' and '1', there are **four types of classification outcomes** namely, class-0 correctly classified as class-0, class-1 correctly classified as class-1, class-0 incorrectly classified as class-1, and class-1 incorrectly classified as class-0. These four types of classification outcomes form the confusion matrix.
- In order to quantify each classified sample into one of these four types of outcomes, a **classification score function** or loss function plays the role of decision making. These **classification loss functions** shall be discussed below.

Classification Accuracy

Illustration



Classification Accuracy

Given a training set consisting of m examples $(\mathbf{x}_i, y_i)$, $i = 1, 2, \dots, m$, where $\mathbf{x}_i \in \mathbb{R}^d$ denotes the i^{th} feature sample, and $y_i \in \{0, 1\}$ denotes the corresponding category indicator or target label. The value y_i can also be viewed as the class associated with $\mathbf{x}_i$.

For the **binary classification** problem, our goal is to determine a predictor g based on the models covered in the previous chapter and a **threshold τ** , utilizing the given training set so that a correct class label prediction can be obtained for unseen data $(\mathbf{x}_j, y_j)$, $j = 1, 2, \dots, n$. An ideal classifier g° is such that $cls(g^\circ(\mathbf{x}_j)) = y_j$ for all $j = 1, 2, \dots, n$, which relates each $\mathbf{x}_j$ to its truth target label y_j via the following labeling or **scoring function**:

$$cls(g(\mathbf{x}_j)) = \begin{cases} 1 & \text{if } g(\mathbf{x}_j) \geq \tau \\ 0 & \text{if } g(\mathbf{x}_j) < \tau \end{cases}, \quad j = 1, 2, \dots, n. \quad (4.7)$$

Classification Accuracy

For notation convenience, we shall indicate a superscript $+$ or $-$ on the variable according to whether it belongs to **class-1 (positive class)** or **class-0 (negative class)**, respectively.

Among the **m learning samples**, suppose the samples $\mathbf{x}_i^+, i = 1, 2, \dots, m^+$, belong to the positive class while the samples $\mathbf{x}_j^-, j = 1, 2, \dots, m^-$, belong to the negative class. In other words, m^+ is the number of training samples $\mathbf{x}_i$ with label $y_i = 1$ and m^- is the number of training samples $\mathbf{x}_i$ with label $y_i = 0$.

Similarly, for **test data set containing n samples**, n^+ is the number of test samples $\mathbf{x}_j$ with label $y_j = 1$ and n^- is the number of test samples $\mathbf{x}_j$ with label $y_j = 0$ for $j = 1, 2, \dots, n$.

Classification Accuracy

Suppose a predictor $\hat{g}$ is chosen (trained), we can choose a threshold τ to classify the test samples into positive class (P) or negative class (N). Based on the given set of test samples, we calculate the following **numbers** for the chosen threshold τ in (4.7):

TP = ‘number of $\mathbf{x}_j$ with $y_j = 1$ and $cls(\hat{g}(\mathbf{x}_j)) = 1$ ’ (true positive)

FN = ‘number of $\mathbf{x}_j$ with $y_j = 1$ but $cls(\hat{g}(\mathbf{x}_j)) = 0$ ’ (false negative)

TN = ‘number of $\mathbf{x}_j$ with $y_j = 0$ and $cls(\hat{g}(\mathbf{x}_j)) = 0$ ’ (true negative)

FP = ‘number of $\mathbf{x}_j$ with $y_j = 0$ but $cls(\hat{g}(\mathbf{x}_j)) = 1$ ’ (false positive)

Classification Accuracy

This forms a contingency table called *confusion matrix*:

Table 1: Confusion matrix for binary problems

Prediction	Ground Truth		
	P	N	
$\hat{P}$	TP	FP	→ Positive predictive value (Precision) $TP/(TP + FP)$
$\hat{N}$	FN	TN	→ Negative predictive value $TN/(FN + TN)$
	Sensitivity (Recall) $TP/(TP + FN)$	Specificity $TN/(FP + TN)$	Accuracy $\frac{TP+TN}{TP+TN+FP+FN}$

Classification Accuracy

The FP and FN are also known as *type I error* and *type II error* respectively.

Mathematically, these numbers can be written as

$$TP = \sum_{j=1}^{n^+} 1_{g(\mathbf{x}_j^+) \geq \tau}, \quad (4.8)$$

$$FN = \sum_{j=1}^{n^+} 1_{g(\mathbf{x}_j^+) < \tau}, \quad (4.9)$$

$$TN = \sum_{j=1}^{n^-} 1_{g(\mathbf{x}_j^-) \leq \tau}, \quad (4.10)$$

$$FP = \sum_{j=1}^{n^-} 1_{g(\mathbf{x}_j^-) > \tau}, \quad (4.11)$$

where $1_{g(\mathbf{x}_j) \geq \tau}$ gives a ‘1’ whenever $g(\mathbf{x}_j) \geq \tau$ and vice versa. These numbers can be normalized according to the population size of each class giving the *true positive rate* ($TPR = TP/n^+$), *true negative rate* ($TNR = TN/n^-$), *false positive rate* ($FPR = FP/n^-$) and *false negative rate* ($FNR = FN/n^+$).

Classification Accuracy

Since $n^+ = TP + FN$, $n^- = TN + FP$ and

$$TPR + FNR = \frac{TP}{n^+} + \frac{FN}{n^+} = \frac{(TP + FN)}{n^+} = 1, \quad (4.12)$$

$$TNR + FPR = \frac{TN}{n^-} + \frac{FP}{n^-} = \frac{(TN + FP)}{n^-} = 1, \quad (4.13)$$

it is sufficient to describe a classifier's performance using two out of the four rates, one from each of the above two equations. For instance, given TPR and TNR, one can derive FNR and FPR respectively based on $(1 - TPR)$ using (4.12) and based on $(1 - TNR)$ using (4.13).

Classification Accuracy

The confusion matrix can be extended to [multi-category problems](#). Suppose the data contains c classes and the number of samples in each class is respectively labelled as $P_1, P_2, \dots, P_c$. Correspondingly, the predictions for these classes are labelled as $P_{\hat{1}}, P_{\hat{2}}, \dots, P_{\hat{c}}$. Then the corresponding truth-prediction pair can be written as $P_{i,\hat{j}}$ where $i, j \in \{1, 2, \dots, c\}$. These truth-prediction pairs can be tabulated as a confusion matrix as follows:

Table 2: Confusion matrix for multi-class problems

Ground Truth	Prediction			
	$P_{\hat{1}}$	$P_{\hat{2}}$	$\dots$	$P_{\hat{c}}$
P_1	$P_{1,\hat{1}}$	$P_{1,\hat{2}}$	$\dots$	$P_{1,\hat{c}}$
P_2	$P_{2,\hat{1}}$	$P_{2,\hat{2}}$	$\dots$	$P_{2,\hat{c}}$
$\vdots$	$\vdots$		$\ddots$	$\vdots$
P_c	$P_{c,\hat{1}}$	$P_{c,\hat{2}}$	$\dots$	$P_{c,\hat{c}}$

Classification Accuracy

When the P 's are used as the number of samples for the corresponding prediction and ground truth, we can read $P_{i,\hat{j}}$ as the number of samples actually belonging to class- i which are predicted as class- $\hat{j}$. The numbers in the diagonal entries thus correspond to those correctly predicted samples and they can be expressed as the complements of sum of off-diagonal entries in each column:

$$P_{i,\hat{i}} = P_{\hat{j}} - \sum_{i=1}^c P_{i,\hat{j}}. \quad (4.14)$$

Similarly, for row wise summation, the following holds:

$$P_{i,\hat{i}} = P_i - \sum_{j=1}^c P_{i,\hat{j}}. \quad (4.15)$$

In terms of [recognition rates](#) or [probability](#) $p_{i,\hat{j}}$ which sums to 1, $\forall j$, (4.14) can be re-written as

$$p_{i,\hat{i}} = 1 - \sum_{i=1}^c p_{i,\hat{j}}. \quad (4.16)$$

Accuracy, Precision and F-measure

In binary classification, the above rates define several frequently used performance measures such as those given in Table 3,

Table 3: Some performance measures for binary classifiers

Performance measures	Definition	$s = 1$	$s = n^-/n^+$
True Positive Rate	$\text{TPR} = \frac{\text{TP}}{n^+}$	—	—
True Negative Rate	$\text{TNR} = \frac{\text{TN}}{n^-}$	—	—
Accuracy	$\frac{\text{TPR} + s \cdot \text{TNR}}{1 + s}$	$\frac{\text{TPR} + \text{TNR}}{2}$	$\frac{\text{TP} + \text{TN}}{n^+ + n^-}$
Precision	$\frac{\text{TPR}}{\text{TPR} + s \cdot \text{FPR}}$	$\frac{\text{TPR}}{\text{TPR} + \text{FPR}}$	$\frac{\text{TP}}{\text{TP} + \text{FP}}$
F-measure	$\frac{2\text{TPR}}{\text{TPR} + s \cdot \text{FPR} + 1}$	$\frac{2\text{TPR}}{\text{TPR} + \text{FPR} + 1}$	$\frac{2\text{TP}}{\text{TP} + \text{FP} + n^+}$

where s is a skewing factor which is frequently taken to be the ratio between the size of negative samples and the size of positive samples (i.e., $s = n^-/n^+$).

Misclassification Error

Apart from the accuracy related measures defined above, the performance of a classification system can also be described using the error measure. A commonly adopted error score function for **misclassification** is a counting function given by

$$err(g, y) = 1_{(g \neq y)}. \quad (4.17)$$

The *misclassification error* or total misclassification counts, when normalized by the number of samples, is a dual of accuracy in two-category problems when $s = n^-/n^+$ in $\frac{TPR+s \cdot TNR}{1+s}$, i.e.,

$$\frac{err}{n} = 1 - Accuracy|_{s=n^-/n^+} = \frac{FP + FN}{TP + TN + FP + FN}. \quad (4.18)$$

For multi-category problems, the misclassification error is the sum of all off-diagonal elements in Table 2. The misclassification error rate is the misclassification error normalized by the total sample size.

Total Error Rate

In binary classification, when the normalization is performed with respect to each pattern class, an error rate known as the *total error rate* can be defined as follows:

$$\text{TER} = \text{FPR} + \text{FNR}. \quad (4.19)$$

Alternatively, the half total error rate $\text{HTER} = (\text{FPR} + \text{FNR})/2$ can be adopted when a normalization between the two categories is considered.

As seen from Table 3, HTER is a *dual* of *accuracy with a unit skewing factor* $s = 1$:

$$\begin{aligned} \text{Accuracy} &= \frac{\text{TPR} + \text{TNR}}{2} = \frac{1}{2}[(1 - \text{FNR}) + (1 - \text{FPR})] \\ &= \frac{1}{2}(2 - \text{TER}) = 1 - \text{HTER}. \end{aligned} \quad (4.20)$$

When $s = n^-/n^+$,

$$\begin{aligned} \text{Accuracy} &= \frac{\text{TP} + \text{TN}}{n^+ + n^-} = \frac{n^+ \left(\frac{\text{TP}}{n^+}\right) + n^- \left(\frac{\text{TN}}{n^-}\right)}{n^+ + n^-} \\ &= \frac{n^+}{n}(1 - \text{FNR}) + \frac{n^-}{n}(1 - \text{FPR}). \end{aligned} \quad (4.21)$$

This shows that accuracy at $s = n^-/n^+$ is related to weighted error rates.

Equal Error Rate

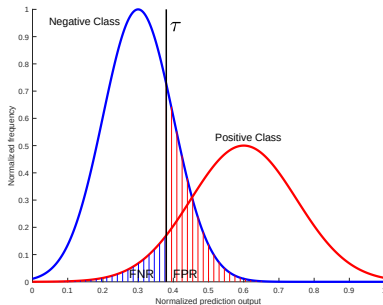
So far we have been relying on a [predetermined threshold value](#) to compute the error and accuracy related measures.

Without loss of generality, suppose the output of binary predictor $\hat{g}$ is normalized within $[0, 1]$. Then, we can [vary the threshold \$\tau\$ within \$\[0, 1\]\$ to generate a series of error rates](#).

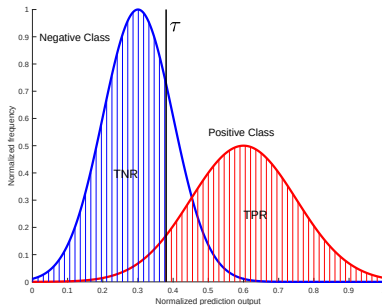
Fig. 1 shows a normalized plot illustrating those corresponding areas to the rates of FP, FN, TP and TN obtained at a threshold value τ for two overlapping distributions of prediction output.

By varying $\tau \in [0, 1]$, these rates (shaded areas) can vary from 0% to 100%. Given two overlapping distributions of prediction output with the mean of positive class higher than the mean of negative class as illustrated in Fig. 1, a typical FPR plot shows a decreasing trend while a FNR plot shows an increasing trend. Fig. 2(a) shows a plot for the FPR and the FNR over variation of threshold τ . In this plot, [the intersection point of FPR and FRR curves is called the *equal error rate* \(EER\)](#).

Equal Error Rate



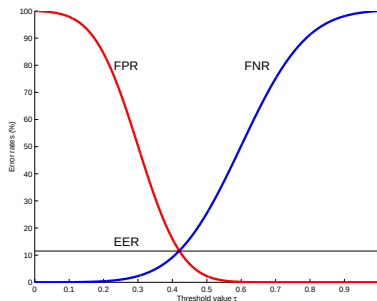
(a) FNR and FPR



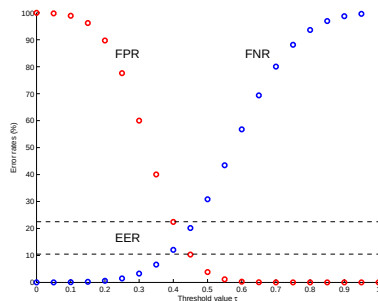
(b) TNR and TPR

Figure 1: Normalized distributions of binary prediction outputs.

Equal Error Rate



(a) EER



(b) EER approximation for discrete data

Figure 2: Error rates versus threshold.

Loss Functions

- Apart from the distance error loss given by $[g(\mathbf{x}_i, \mathbf{w}) - y_i]^2$ in regression and the binary counting loss given by (4.7) which is also known as ‘0-1’ loss, several other types of loss function are also available for classification.
- Suppose we have a set of binary classification learning samples $\{\mathbf{x}_i, y_i\}_{i=1}^m$ with $y_i \in \{-1, +1\}$. Let $g(\mathbf{x}_i)$ be the output of the classifier that we built. Then, based on the *margin* given by $\mu_i = y_i g(\mathbf{x}_i)$, a learning *loss* term indicating the success or failure of classifying each sample can be defined as

$$L_{\text{margin}}(y_i, g(\mathbf{x}_i)) = y_i g(\mathbf{x}_i) = \mu_i, \quad i = 1, \dots, m, \quad (4.22)$$

where a *positive margin* with y_i and $g(\mathbf{x}_i)$ having the same sign indicates a *correct classification* and a *negative margin* indicates an *incorrect classification*. Here, we note that an *implicit zero threshold* has been assumed between the positive and negative values.

Loss Functions

- Since the loss term can be applied to both the training set and the testing set, we shall omit the sample indexing for the margin term (i.e., we shall use μ instead of μ_i) in the sequel. When the margin μ has been adopted as an input to the '0-1' loss function, the loss term can be written as

$$L_{01}(\mu) = \begin{cases} 0 & \text{if } \mu \geq 0, \text{ meaning 0 loss} \\ 1 & \text{if } \mu < 0. \text{ meaning 1 loss} \end{cases} \quad (4.23)$$

- A squared loss function is often used in the sum of squared errors (4.24) to form the minimization objective or cost function:

$$J(\mathbf{w}) = \sum_{i=1}^m (g(\mathbf{x}_i) - y_i)^2. \quad (4.24)$$

Loss Functions

- To deal with the case of different costs associated with different misclassifications, a weighting factor ω_i [Lowe and Webb, 1990, Bishop, 1995] can be incorporated for each training sample giving a **weighted sum of squares**:

$$J(\mathbf{w}) = \sum_{i=1}^m \sum_{k=1}^c \omega_i (g_k(\mathbf{x}_i) - y_{k,i})^2. \quad (4.25)$$

- Apart from the square loss, another popular choice of loss term **for classification** is the **hinge loss** adopted in the Support Vector Machines (SVM):

$$L_{\text{hinge}}(\mu) = \max(0, 1 - yg(\mathbf{x})), \quad (4.26)$$

where $y \in \{-1, +1\}$ and $g \in \mathbb{R}$.

Loss Functions

- Expressed in terms of the margin μ , a *logistic loss* can be written as

$$L_{\text{logistic}}(\mu) = \log(1 + e^{-\mu}) = \log(1 + e^{-yg(\mathbf{x})}), \quad (4.27)$$

or

$$L_{\text{logistic}}(\mu) = (\log 2)^{-1} \log(1 + e^{-\mu}), \quad (4.28)$$

where the constant term $(\log 2)^{-1}$ ensures that all losses for classification equal 1 for $g(\mathbf{x}) = 0$. The logistic loss is used in the LogitBoost algorithm. A variant of the above, the *exponential loss* which is used in the AdaBoost algorithm, is given by

$$L_{\text{exp}}(\mu) = \exp(-\mu) = \exp(-yg(\mathbf{x})). \quad (4.29)$$

Loss Functions

- In *logistic regression*, a linear parametric model is used for fitting the *log-odds* function. For instance, fitting the model $g(\mathbf{x}) = \alpha_0 + \alpha^T \mathbf{x}$ to the *log-odds* gives

$$\begin{aligned} \log(p/(1-p)) &= g(\mathbf{x}) \\ \frac{p}{1-p} &= e^{g(\mathbf{x})} \end{aligned} \quad (4.30)$$

which gives the predicted probability distribution of the model output response given by

$$p = \sigma(g(\mathbf{x})) = \frac{1}{1 + e^{-g(\mathbf{x})}} = \frac{e^{g(\mathbf{x})}}{1 + e^{g(\mathbf{x})}}. \quad (4.31)$$

The term $\sigma(\cdot) = \frac{1}{1+e^{-\cdot}}$ is known as the *logistic function* or *sigmoid function*.

Loss Functions

- The *binary log-likelihood* for maximization is defined as

$$L_{\log\text{-likelihood}}(p) = y \log p + (1 - y) \log(1 - p) \quad (4.32)$$

with its dual form for minimization as **binary cross-entropy**:

$$L_{\text{cross-entropy}}(p) = -y \log p - (1 - y) \log(1 - p). \quad (4.33)$$

- For **multi-category** problems with c classes, the multi-class cross-entropy is defined as:

$$L_{\text{cross-entropy}}(p) = - \sum_{i=1}^c y_i \log(p_i), \quad (4.34)$$

where y_i and p_i are respectively the target category label and the prediction probability of the i th class.

Loss Functions

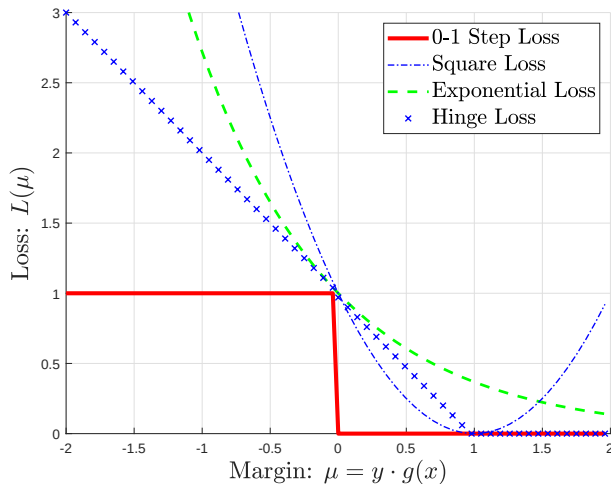


Figure 3: Loss functions: margin-loss plot

Remarks:

- Fig. 3 presents several plots of the discussed loss functions.
- Except for square loss, these loss functions primarily target binary classification.
- The 0-1 step loss is seldom used directly for training classifiers because its gradient is either zero or undefined. Instead, other methods employ a **smooth, continuous surrogate objective** to facilitate optimization during the learning process.

Regularization Term

The **regularization** term in a regression or classification cost or objective function can be considered a loss term which accounts for **penalizing the size of parameters**. In general, such a cost function can be written as composing of two loss components namely, $L(\cdot)$ and $R(\cdot)$ as follows:

$$J(\mathbf{w}) = \sum_{i=1}^m L(\mu_i(\mathbf{w})) + \lambda R(\mathbf{w}), \quad (4.35)$$

where λ is a scalar regularization weighting factor, $\mathbf{w}$ denotes the adjustable weight parameter vector of the classifier g , and μ_i is the margin loss term for each sample given by $y_i g(\mathbf{x}_i, \mathbf{w})$.

Regularization Term

- $R_2 = \frac{1}{2} \|\mathbf{w}\|_2^2$, ridge regression,
- $R_1 = \sum_i |w_i|$, lasso,
- $R_0 = |\{i : w_i \neq 0\}|$, subset selection.

In general, the above regularization terms can be interpreted as members of the generic family:

$$R_q = \left(\sum_i |w_i|^q \right)^{\frac{1}{q}}. \quad (4.36)$$

Regularization Term

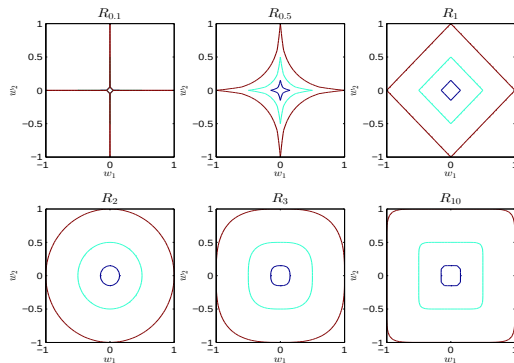


Figure 4: Regularization functions ($R_{0.1}$, $R_{0.5}$, R_1 , R_2 , R_3 , R_{10}) for two-dimensional w . The outer-most contour indicates one unit distance from the origin.

Ranking and Operating Characteristics

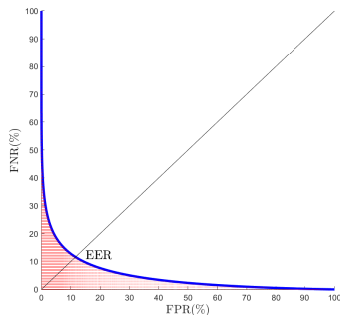
Apart from the above types of cost functions which measures either the magnitude or distance of error for regression or the count of correctly or incorrectly classified samples, there is another type of performance criterion which is based on the **ranking order of comparison**.

The *receiver operating characteristic* curve provides such as measure of ranking order for **every operating threshold** of the classifier. In the following, several classification cost functions are presented with the relationship among them.

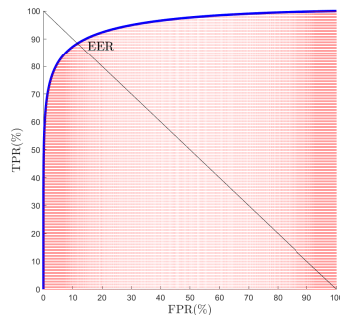
Receiver Operating Characteristics

- Instead of plotting two separate curves, $\text{FNR}(\tau)$ mentioned can be plotted over $\text{FPR}(\tau)$ for the entire operating range of threshold τ values to obtain a single curve (Fig. 5(a)) in binary classification tasks.
- This plot is called a *receiver operating characteristic (ROC) curve*. Here, the more bending is the curve is towards the origin, the better is the performance of a classifier.
- Alternatively, $\text{TPR}(\tau) = 1 - \text{FNR}(\tau)$ can be plotted over $\text{FPR}(\tau)$ (Fig. 5(b)) where the more bow is the curve towards the upper left corner, the better is the performance of a classifier.

Receiver Operating Characteristics



(a) FNR versus FPR or
(1-Sensitivity) versus (1-Specificity)



(b) TPR versus FPR or
Sensitivity versus (1-Specificity)

Figure 5: ROC curves.

Area Under the ROC

- Since the ROC provides a range of performance values instead of a single value, the Area Under the ROC Curve (AUC) is a natural choice as the performance measure for overall classifier comparison.
- The AUC can be viewed as the probability that the score of the positive class observation is greater than that of the negative class observation.
- In the discrete case, this probability (and hence AUC) can be computed via the fraction of positive-negative pairs that are ranked correctly.

Area Under the ROC

- Consider a set of binary training data indexed by $i = 1, 2, \dots, m^+$ for positive class and $j = 1, 2, \dots, m^-$ for negative class. Let us denote $\xi_{ij} = g(\mathbf{x}_i^+) - g(\mathbf{x}_j^-)$ which is the difference in the value of the predictor between a positive sample $\mathbf{x}_i^+$ and a negative sample $\mathbf{x}_j^-$. Thus, $\xi_{ij} > 0$ indicates that the positive sample $\mathbf{x}_i^+$ and the negative sample $\mathbf{x}_j^-$ are correctly ranked by the predictor g . Consider also a Heaviside step function given by

$$u(\xi) = \begin{cases} 1, & \text{if } \xi > 0, \\ 0.5 & \text{if } \xi = 0, \\ 0, & \text{if } \xi < 0. \end{cases} \quad (4.37)$$

- The AUC for m training examples can then be expressed as

$$\text{AUC} = \frac{1}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} u(\xi_{ij}) \quad , \quad (4.38)$$

which is the fraction of positive-negative pairs that are ranked correctly by predictor g . In other words, what matters for AUC is the score order but not the score value itself.

Lorenz Curve and Gini Coefficient

- In economics, the Lorenz curve is a graphical plot to represent the distribution of wealth. Derived from the Lorenz curve, the Gini coefficient, which is also called the [Gini index](#), is a linear re-scaling of AUC given by $2 \times \text{AUC} - 1$.
- As shown in Fig. 6, the [Gini coefficient](#) is the ratio of areas given by $G = A / (A + B)$. Since $(A + B) = 0.5$, then half of this area ratio (i.e., A itself and not divided by $(A + B)$) is equal to $\text{AUC} - 0.5$. In economic and econometric terms, a Gini coefficient of zero ($\text{AUC} = 0.5$) means that income is evenly distributed. This can be interpreted as having a non-discriminative classifier when $\text{AUC} = 0.5$.

Ranking and Operating Characteristics

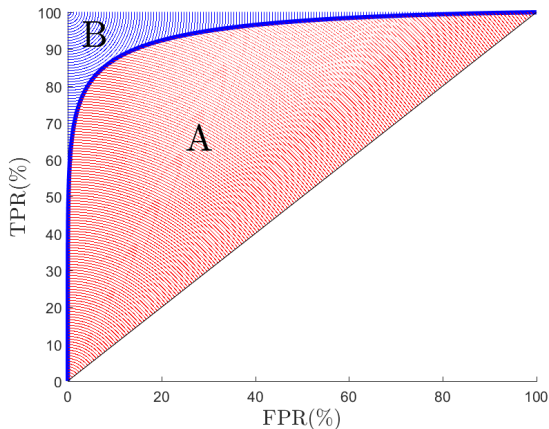


Figure 6: Sub-areas on the ROC plot.

Relationships Among LSE, TER and AUC

The LSE, Acc, TER and AUC cost functions for training are listed together below for comparison:

$$\text{MSE : } J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m (g(\mathbf{x}_i, \mathbf{w}) - y_i)^2, \quad (\text{regression}), \quad (4.39)$$

$$\text{MCE : } J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m (u(y_i g(\mathbf{x}_i, \mathbf{w})) - 1)^2, \quad (\text{classification}), \quad (4.40)$$

$$\text{TER : } J(\mathbf{w}) = \frac{1}{m^-} \sum_{i=1}^{m^-} u(g(\mathbf{x}_i^-, \mathbf{w}) - y_i^-) + \frac{1}{m^+} \sum_{in=1}^{m^+} u(y_i^+ - g(\mathbf{x}_i^+, \mathbf{w})),$$

(classification) (4.41)

$$\text{AUC : } J(\mathbf{w}) = \frac{1}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} u(g(\mathbf{x}_i^+, \mathbf{w}) - g(\mathbf{x}_j^-, \mathbf{w})),$$

(classification ranking order), (4.42)

where

$$u(\xi) = \begin{cases} 1, & \text{if } \xi > 0, \\ 0.5 & \text{if } \xi = 0, \\ 0, & \text{if } \xi < 0. \end{cases}$$

Summary

- The need for different loss functions
- Regression
 - * Use of **squared error loss**
 - * to form **MSE** or **SSE** cost function
- Classification
 - * Use of **0-1 loss** (by thresholding the learned model output $\hat{g}(\mathbf{x})$)
 - * Use of **smooth, continuous surrogate loss and objective** to facilitate optimization during the learning process.
 - * With consideration of different types of true category (TP, TN) and error (FP, FN) to form the **confusion matrix**
 - * Subsequently form the learning cost function quantified by either **FPR-FNR** pair (or TPR-TNR pair).
 - * ROC, AUC cost function.
- Relationships among LSE, TER and AUC

Thank you!



Bishop, C. M. (1995).

Neural Networks for Pattern Recognition.

Oxford University Press Inc., New York.



Lowe, D. and Webb, A. (1990).

Exploiting prior knowledge in network optimization: an illustration from medical prognosis.

Network: Computation in Neural Systems, 1(3):299 – 323.